

Operationalizing an AWS ML Project

Project Directory Structure

```
operationalizing-ml-on-sagemaker/
  IaC/                                     # Infrastructure as Code (Terraform)
    dev/
      artefacts/
        lambdas/
          udacity-cjmm-exec-inf-breaddog-cls.zip
        lambdas/
          udacity-cjmm-exec-inf-breaddog-cls/
            index.py                       # Lambda function source code
      extra-policies/
        lambda/
          lambda-udacity-cjmm-exec-inf-breaddog-cls-dev.json # IAM policy for Lambda
      main.tf                             # Main Terraform configuration
      terraform.tf                         # Terraform provider and backend settings
      variables.tf                         # Terraform input variables
  screenshots/                            # Project evidence screenshots
    01-a_sagemaker-notebook-instance.png
    01-b_s3-bucket.png
    01-c_endpoint-1_detail_single-instance-training.png
    01-c_endpoint-1_detail-multi-instance-training.png
    01-c_endpoint-1_single-instance-training.png
    02-a_ec2-instance.png
    02-b_ec2-terminal-ssh.png
    04-a_lambda-function-testing.png
    04-b_IAM-role.png
    04-b_IAM-role-policy-detail.png
    05_lambda-concurrency.png
    all_endpoints_including_deployed_byterraform.png
    models-deployed.png
  ec2train1.py                            # EC2 training script
  hpo.py                                  # SageMaker hyperparameter tuning script
  infernce2.py                            # Inference script for endpoint deployment
  lab.jpg                                 # Sample test image
  lamdafunction.py                        # Lambda function for invoking the endpoint
  train_and_deploy-solution.ipynb        # Main Jupyter notebook
  train_and_deploy-solution.html         # HTML export of the notebook
  README.md                              # Project documentation (this file)
```

Step 1: Training and deployment

Initial Setup

First, the project template `starter.zip` was downloaded and extracted into the main directory of the git repository. Next, the main Infrastructure as Code file was created in Terraform to provision a SageMaker Notebook Instance with the `ml.t3.medium` instance type. Since we are not performing model training within the notebook instance itself, this instance type is sufficient given the cost optimization it provides — it is one of the cheapest options and the default instance type even in SageMaker Domain Studio projects. With this instance, we proceeded to run the model training notebook.

A screenshot of the notebook instance:

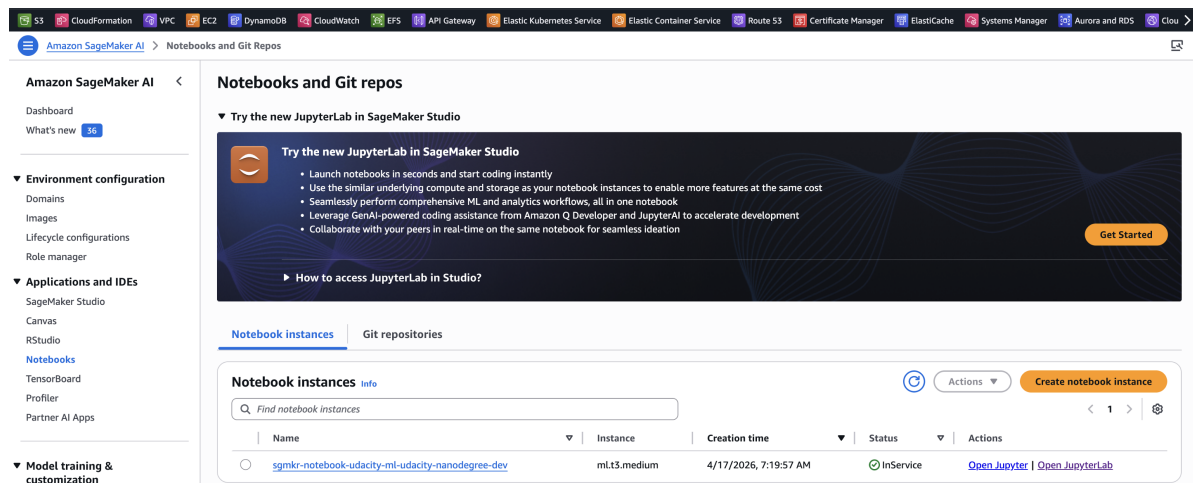


Figure 1: Notebook Instance

`starter.zip` archive contents:

```
starter.zip
  ec2train1.py
  hpo.py
  infernce2.py
  lab.jpg
  lamdafunction.py
  train_and_deploy-solution.ipynb
```

Finally, to synchronize the project environment (git repository) with the SageMaker Notebook Instance, the repository was cloned into the notebook instance by running `git clone https://github.com/cjmunoz131/udacity-aws-ml-nanodegree.git`.

Download data to an S3 bucket

To download the data to S3, an S3 bucket was first created as a Terraform resource in the `main.tf` template. After redeploying all resources defined in `main.tf`, the SageMaker Notebook Instance was used to run the notebook cell that downloaded and uploaded the image dataset to S3.

A screenshot of the created bucket:

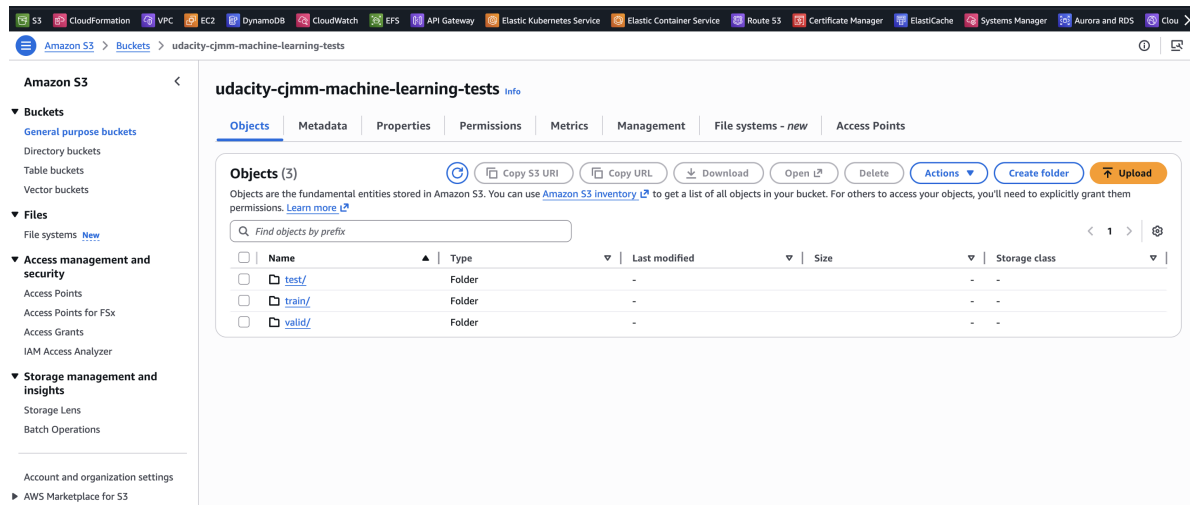


Figure 2: S3 bucket

Training and Deployment (Single Instance Training)

For single-instance training and deployment, cells 14 through 16 of the `train_and_deploy-solution.ipynb` notebook were executed. I created a tuning job with an `m1.m5.xlarge` instance type, `max_jobs=2` and `max_parallel_jobs=1`, where each job took approximately 20 minutes for a total of 40 minutes. The best hyperparameters found were: `{'batch_size': 128, 'learning_rate': '0.006065489818368382'}`

Then, I ran the machine learning model training using the best hyperparameters found by the tuner. I used an `m1.m5.xlarge` instance, which provided sufficient computational power for training while optimizing cost.

For the **Deployment** section, I kept the `m1.m5.large` instance type as it was sufficient for the inference tasks through the Lambda function invocations.

To verify that the deployment was working correctly, I executed a request with the following dictionary: `{ "url": "https://s3.amazonaws.com/cdn-origin-etr.akc.org/wp-`

content/uploads/2017/11/20113314/Carolina-Dog-standing-outdoors.jpg" } and
obtained the following resulting vector:

```
[-5.946874141693115,  
 -1.21341073513031,  
 -5.605361461639404,  
 -1.325074553489685,  
 -1.3399231433868408,  
 -2.896413803100586,  
 -2.1387393474578857,  
 -0.1960349678993225,  
 -5.652327537536621,  
 -0.8442195653915405,  
 0.13509000837802887,  
 -2.551924228668213,  
 -2.085651397705078,  
 -0.5827327370643616,  
 -4.500021934509277,  
 -2.4181454181671143,  
 -2.8068978786468506,  
 -2.466200113296509,  
 -3.2664778232574463,  
 0.3240165710449219,  
 -2.4202916622161865,  
 0.10326236486434937,  
 -8.082221984863281,  
 -1.789431095123291,  
 -1.8198843002319336,  
 -8.243498802185059,  
 -4.617023944854736,  
 -4.80702018737793,  
 -2.20324969291687,  
 -2.4415276050567627,  
 -2.0191564559936523,  
 -4.643577575683594,  
 -4.673723220825195,  
 -5.069280624389648,  
 -4.878660678863525,  
 -2.1586358547210693,  
 -6.963231563568115,  
 -3.5415706634521484,  
 -2.761871099472046,  
 -3.593057870864868,
```

-4.3033671379089355,
-1.8653879165649414,
-1.6894434690475464,
-4.6258544921875,
-1.6334173679351807,
-5.613066673278809,
-3.7227866649627686,
-1.2190767526626587,
-1.1653605699539185,
-3.7402749061584473,
0.31676146388053894,
-5.48195743560791,
-2.795227289199829,
-1.085308313369751,
-5.123355388641357,
-1.7913445234298706,
-8.002077102661133,
-3.436387300491333,
-5.631474494934082,
-3.2476205825805664,
-4.956148624420166,
-5.04777193069458,
-5.328779697418213,
-7.433752536773682,
-3.8618571758270264,
-6.392577648162842,
-1.6519601345062256,
-4.480065822601318,
-4.290234088897705,
-5.907329559326172,
-0.41159453988075256,
-3.487931251525879,
-6.221728324890137,
-6.159914970397949,
-3.3440349102020264,
-0.5000829696655273,
-9.717192649841309,
-4.3503851890563965,
-1.407018780708313,
-3.4670796394348145,
-0.9024475812911987,
-1.7029597759246826,

-0.7186862826347351,
0.20136758685112,
-4.1870245933532715,
-5.114702224731445,
-3.236370801925659,
-4.702022075653076,
-2.8675785064697266,
-3.3546535968780518,
-5.890472888946533,
0.19747142493724823,
-2.322925329208374,
-3.623960256576538,
-0.06440316140651703,
-1.7555432319641113,
-3.5951035022735596,
-1.2597811222076416,
-4.16080904006958,
-3.045992374420166,
-4.741650104522705,
-4.280559062957764,
-5.500641345977783,
-5.760117053985596,
-4.470567226409912,
-1.7140462398529053,
-1.5557782649993896,
-0.6335992813110352,
0.33968138694763184,
-1.1242530345916748,
-1.3373439311981201,
-1.579911231994629,
-0.6537904739379883,
-3.2389349937438965,
-2.709717035293579,
-2.0316309928894043,
-4.217432498931885,
-1.5741908550262451,
-4.0415730476379395,
0.6823874115943909,
-3.4828333854675293,
-4.87647008895874,
-4.162093639373779,
-1.3526136875152588,

```
-4.711954116821289,
-3.440518379211426,
-4.053915023803711,
-0.3949832320213318,
-5.573906421661377,
-4.367041110992432,
-5.850455284118652,
-1.8168292045593262,
-6.5642571449279785]
```

The endpoint name is 'pytorch-inference-2026-04-17-21-54-53-925' and is shown in the following screenshot:

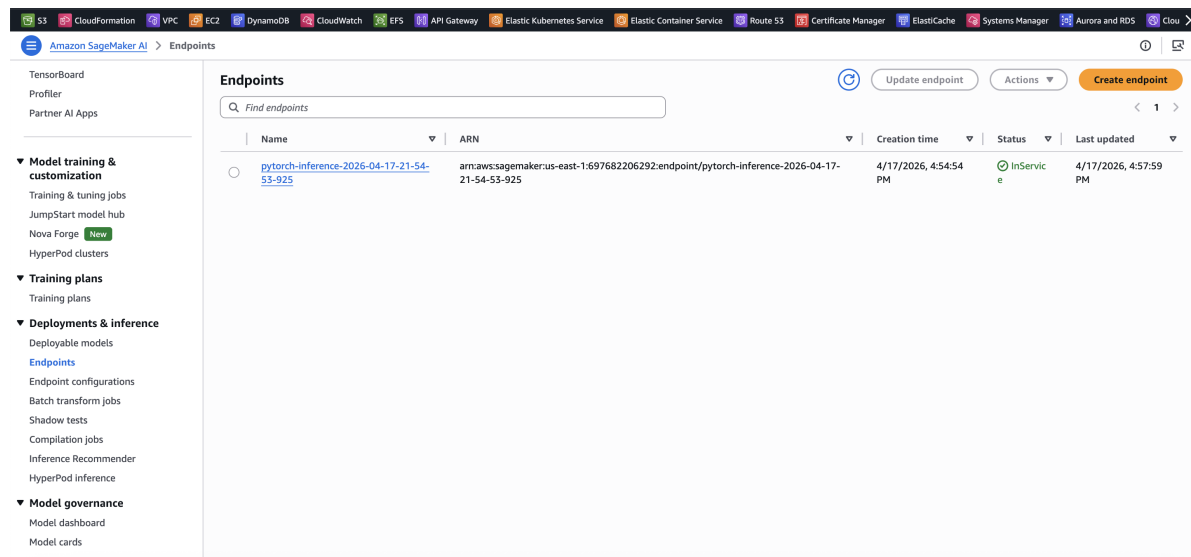


Figure 3: Endpoint - Single Instance

Training and Deployment (Multi-instance training)

I created a multi-instance training job with `instance_count=4` to run 4 instances simultaneously.

```
estimator_multi_instance = PyTorch(
    ...,
    instance_count = 4,
    ...
)
```

Then, the remaining notebook cells were executed to deploy a new endpoint 'pytorch-inference-2026-04-17-23-33-53-853', which is shown in the following screenshot:

The screenshot displays the Amazon SageMaker console interface. The left sidebar shows navigation options under 'Environment configuration', 'Applications and IDEs', 'Model training & customization', and 'Deployments & inference'. The main panel shows the configuration for a new endpoint named 'pytorch-inference-2026-04-17-23-33-53-853'.

Endpoint summary

Name pytorch-inference-2026-04-17-23-33-53-853	Status InService	Type Real-time
ARN arn:aws:sagemaker:us-east-1:697682206292:endpoint/pytorch-inference-2026-04-17-23-33-53-853	Creation time Fri Apr 17 2026 18:33:54 GMT-0500 (hora estándar de Perú)	Last updated Fri Apr 17 2026 18:36:55 GMT-0500 (hora estándar de Perú)
URL https://runtime.sagemaker.us-east-1.amazonaws.com/endpoints/pytorch-inference-2026-04-17-23-33-53-853/invocations	Model container logs /aws/sagemaker/endpoints/pytorch-inference-2026-04-17-23-33-53-853	Alarms 0 alarms

Data capture settings

Enable data capture No	Current sampling percentage (%) -	S3 location to store data collected -	Data capture status -
---------------------------	--------------------------------------	--	--------------------------

Endpoint runtime settings

	Variant name	Current weight	Desired weight	Elastic Inference	Instance type	Current instance count	Desired instance count	Instance min
☐	AllTraffic	1	1	-	mLm5.large	1	1	-

Figure 4: Endpoint - Multi-instance

Finally, I re-ran the inference test cells in the notebook and obtained the following resulting vector:

```
[ -12.12432861328125,
-6.4124016761779785,
-3.7269539833068848,
1.703340768814087,
-4.8407816886901855,
-10.343329429626465,
-0.4081711769104004,
0.9049491882324219,
-9.276532173156738,
0.30977869033813477,
0.44670745730400085,
-6.858445167541504,
-2.987046241760254,
2.7408666610717773,
-6.857962131500244,
-3.4018287658691406,
-13.97025203704834,
-3.422393560409546,
```



```

-3.6992363929748535,
3.8540568351745605,
-7.258420944213867,
-3.0604422092437744,
-13.17885971069336,
-9.634763717651367,
-4.850439071655273,
...
-6.430561065673828,
-11.57479476928711,
-10.314143180847168,
-0.5215942859649658,
-8.82612133026123]

```

Step 2: EC2 Training

For this EC2 training step, I used an [m5.2xlarge](#) instance type, which provides greater computational power while keeping the cost low enough for the training task. This was the same instance type used in the [breed-dog-image-classification-pjt](#) project, which yielded good training results.

The following table shows the SageMaker training instance options considered for this project, along with their on-demand pricing (US East - N. Virginia region) and benchmark results from a 1-epoch training run. Prices verified as of April 2026 from [AWS SageMaker AI Pricing](#) and [Holori SageMaker Calculator](#).

compute instance	vCPUs	RAM (GiB)	GPU	cost/hour (USD)
ml.m5.large	2	8	—	0.115
ml.m5.xlarge	4	16	—	0.230
ml.m5.2xlarge	8	32	—	0.461
ml.m5.4xlarge	16	64	—	0.922
ml.c4.4xlarge	16	30	—	0.955
ml.p2.xlarge	4	61	1x K80 (12 GB)	1.125
ml.p3.2xlarge	8	61	1x V100 (16 GB)	3.825

compute instance	vCPUs	RAM (GiB)	GPU	cost/hour (USD)
ml.g4dn.12xlarge	48	192	4x T4 (64 GB)	4.890

As a training image, I used [Deep Learning AMI \(Amazon Linux 2\) Version 57.0 - ami-09570605cb6ed4f72](#) to train the model.

The Terraform specification used to create the EC2 instance was:

```
module "machine-learning-training-instance-ec2-module" {
  providers = {
    aws.main = aws.account1
  }
  source = "git@github.com:cjmunoz131/.../aws-app-compute-virtual-machine-ec2"

  project      = var.project
  environment = terraform.workspace

  instances_config = {
    # Servidor de aplicación
    app_server = {
      iam_instance_profile = aws_iam_instance_profile.this.name
      associate_public_ip_address = true
      ami                    = "ami-09570605cb6ed4f72"
      instance_type         = "m5.2xlarge"
      subnet_id             = module.aws_networking_base_vpc_layer_module.public_subnet_id_list[0]

      security = {
        security_group_ids = [aws_security_group.client.id]
        key_name           = aws_key_pair.ec2_key_pair.key_name
      }

      root_block_device = {
        volume_size = 100
        volume_type = "gp3"
        encrypted   = true
      }

      additional_volumes = {
        data = {
          device_name = "/dev/sdf"
        }
      }
    }
  }
}
```

```

size           = 20
type           = "gp3"
encrypted      = true
}
}
}
}
}
}

```

Screenshot of the created instance:

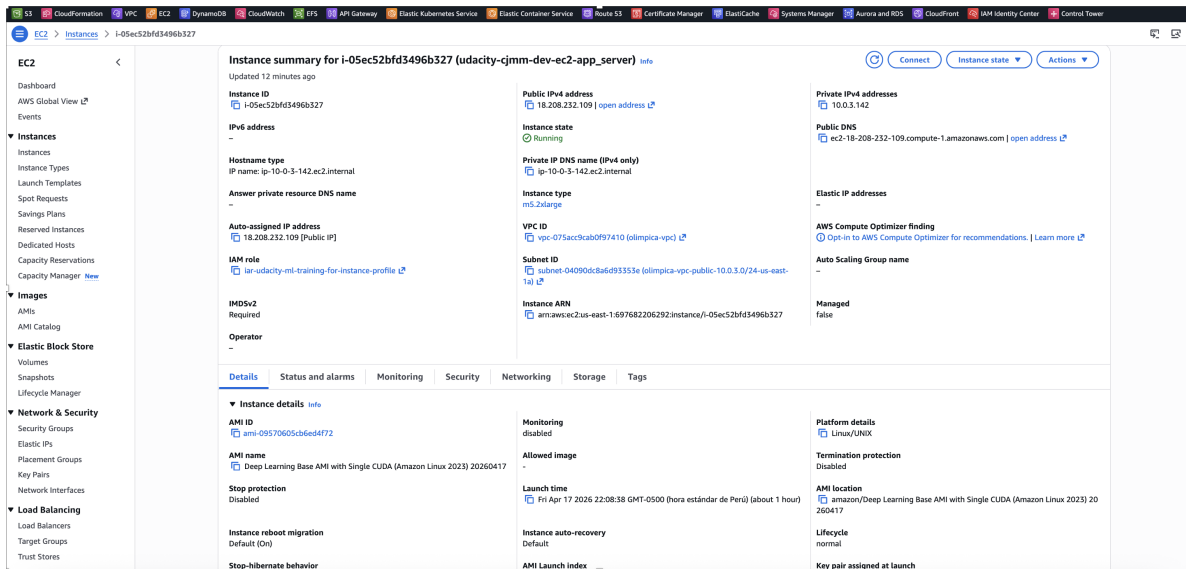


Figure 5: EC2 Instance

Then I used ssh to connect to the instance:

```
ssh -i ~/.ssh/s2smulticloud-kp.pem ec2-user@10.0.3.232
```

Or system manager session manager to connect to the EC2 instance using ssm-user user.

Download the data and create model output directory:

```

wget https://s3-us-west-1.amazonaws.com/udacity-aind/dog-project/dogImages.zip
unzip dogImages.zip
mkdir TrainedModels

```

Install libraries:

```
python3 -m pip install numpy torch torchvision tqdm Pillow
```

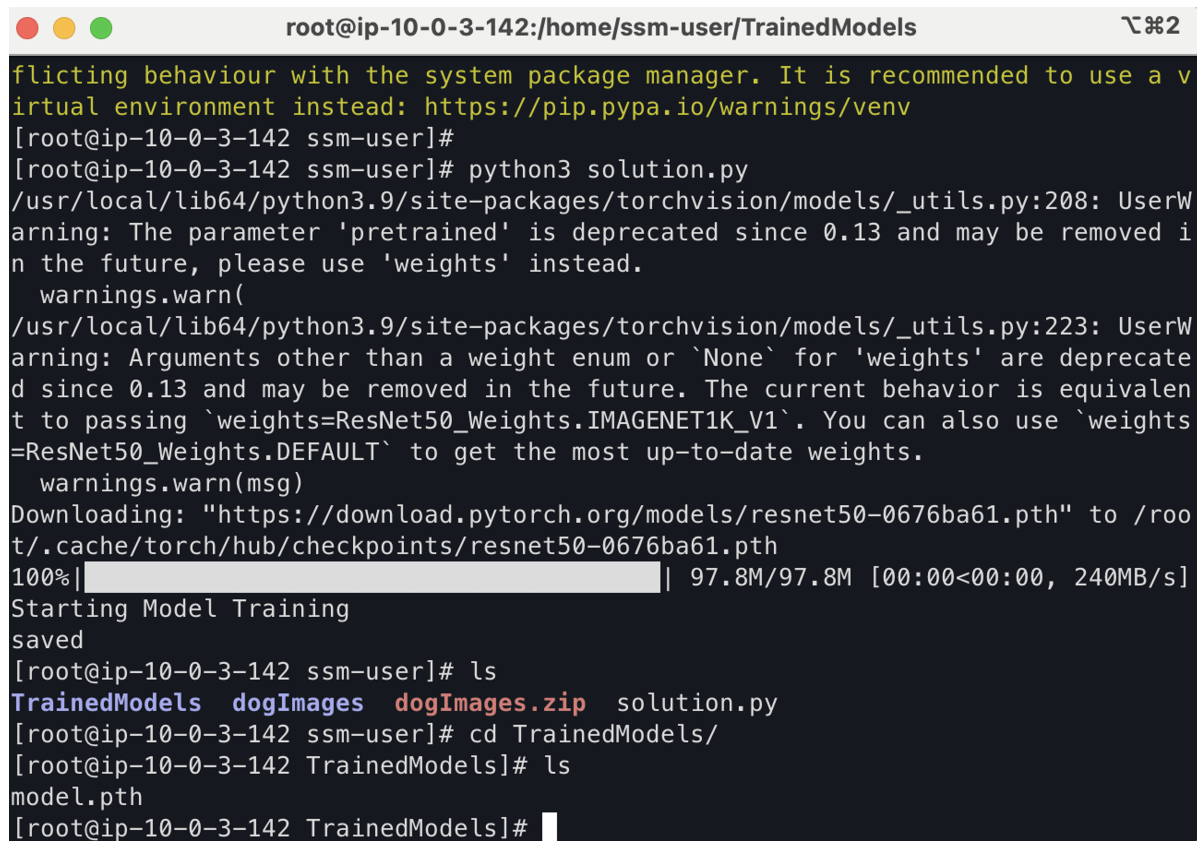
Paste the contents of `ec2train1.py` inside `solution.py` on the machine

```
vim solution.py
```

Train the model

```
python3 solution.py
```

Screenshot of final model training step in terminal:



```
root@ip-10-0-3-142:/home/ssm-user/TrainedModels ㄿ%2
flifting behaviour with the system package manager. It is recommended to use a v
irtual environment instead: https://pip.pypa.io/warnings/venv
[root@ip-10-0-3-142 ssm-user]#
[root@ip-10-0-3-142 ssm-user]# python3 solution.py
/usr/local/lib64/python3.9/site-packages/torchvision/models/_utils.py:208: UserW
arning: The parameter 'pretrained' is deprecated since 0.13 and may be removed i
n the future, please use 'weights' instead.
  warnings.warn(
/usr/local/lib64/python3.9/site-packages/torchvision/models/_utils.py:223: UserW
arning: Arguments other than a weight enum or `None` for 'weights' are deprecate
d since 0.13 and may be removed in the future. The current behavior is equivalen
t to passing `weights=ResNet50_Weights.IMAGENET1K_V1`. You can also use `weights
=ResNet50_Weights.DEFAULT` to get the most up-to-date weights.
  warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/resnet50-0676ba61.pth" to /roo
t/.cache/torch/hub/checkpoints/resnet50-0676ba61.pth
100%|████████████████████████████████████████| 97.8M/97.8M [00:00<00:00, 240MB/s]
Starting Model Training
saved
[root@ip-10-0-3-142 ssm-user]# ls
TrainedModels  dogImages  dogImages.zip  solution.py
[root@ip-10-0-3-142 ssm-user]# cd TrainedModels/
[root@ip-10-0-3-142 TrainedModels]# ls
model.pth
[root@ip-10-0-3-142 TrainedModels]#
```

Figure 6: EC2 Terminal

Difference Between EC2 Training Code and the Code used in Sagemaker

For EC2 training with `ec2train1.py`, an `m1.m5.2xlarge` instance type was selected to provide greater computational power, considering that training runs directly on the same instance rather than on a separate instance spawned by a SageMaker training job from the Notebook Instance. The training job copies the script `hpo.py` to perform the entire training process.

Major differences: - On the EC2 instance, training was performed by executing the script `ec2train1.py` directly from the command line, where all training runs locally. On the SageMaker Notebook Instance, training is carried out through a training job in which an estimator is defined with specific training parameters and instance configurations that ultimately perform the training. The SageMaker notebook used for this task is `train_and_deploy_solution.ipynb`. - The hyperparameters are explicitly passed to the script `hpo.py`, and some parameters such as `data`, `model_dir`, and `output_dir` are obtained from the instance environment variables.

`hpo.py` argument parsing:

```
parser.add_argument('--learning_rate', type=float)
parser.add_argument('--batch_size', type=int)
parser.add_argument('--data', type=str, default=os.environ['SM_CHANNEL_TRAINING'])
parser.add_argument('--model_dir', type=str, default=os.environ['SM_MODEL_DIR'])
parser.add_argument('--output_dir', type=str, default=os.environ['SM_OUTPUT_DATA_DIR'])
```

Instance environment variables:

```
SM_USER_ARGS=["--batch_size","128","--learning_rate","0.006065489818368382"]
SM_HPS={"batch_size":128,"learning_rate":"0.006065489818368382"}
SM_HP_BATCH_SIZE=128
SM_HP_LEARNING_RATE=0.006065489818368382
SM_CHANNEL_TRAINING=/opt/ml/input/data/training
SM_MODEL_DIR=/opt/ml/model
SM_OUTPUT_DIR=/opt/ml/output
SM_USER_ENTRY_POINT=hpo.py
```

In `ec2train1.py`, the hyperparameters are included in the script.

```
batch_size=2
learning_rate=1e-4
```

`ec2train1.py` execution command:

```
python3 ec2train1.py
```

- In SageMaker, the trained model `model.pth` is saved to the implicit path `SM_MODEL_DIR=/opt/ml/model`, which is then compressed and uploaded to S3 as a model artifact registered in the SageMaker Model Registry. In contrast, when training on an EC2 instance, the model is saved locally inside `./TrainedModels` and the user is responsible for moving the model to persistent storage for later use. Additionally, the model is not governed by the SageMaker Model Registry as it would be when training through SageMaker training jobs.

Step 3: Setting up a Lambda function

A Lambda function was provisioned for the dog breed classification inference process using the script `lamdafunction.py`. This Lambda function was defined as a Terraform resource in `main.tf` with the following default permissions: `AWSLambdaBasicExecutionRole`, `AWSLambdaVPCLAccessExecutionRole`, `AWSXRayDaemonWriteAccess`, and `CloudWatch-LambdaInsightsExecutionRolePolicy`. The script `lamdafunction.py` was updated to include `endpoint_Name='pytorch-inference-2026-04-17-21-54-53-925'`.

The function only accepts image URL passed as a request dictionary with content type `application/json`. The request dictionary has the format `{'url': 'http://website.com/image-url.ext'}`

The Lambda function invokes the endpoint `endpoint_Name='pytorch-inference-2026-04-17-21-54-53-925'`, passing the request dictionary in the body and setting the content-type to `application/json`. After configuring the additional permissions, the endpoint returns the predictions in the response body.

Step 4: Lambda Security and Testing

In this step, the `AmazonSageMakerFullAccess` policy was attached so the Lambda function could invoke the inference endpoint.

Screenshot of the IAM role used to execute the lambda function:

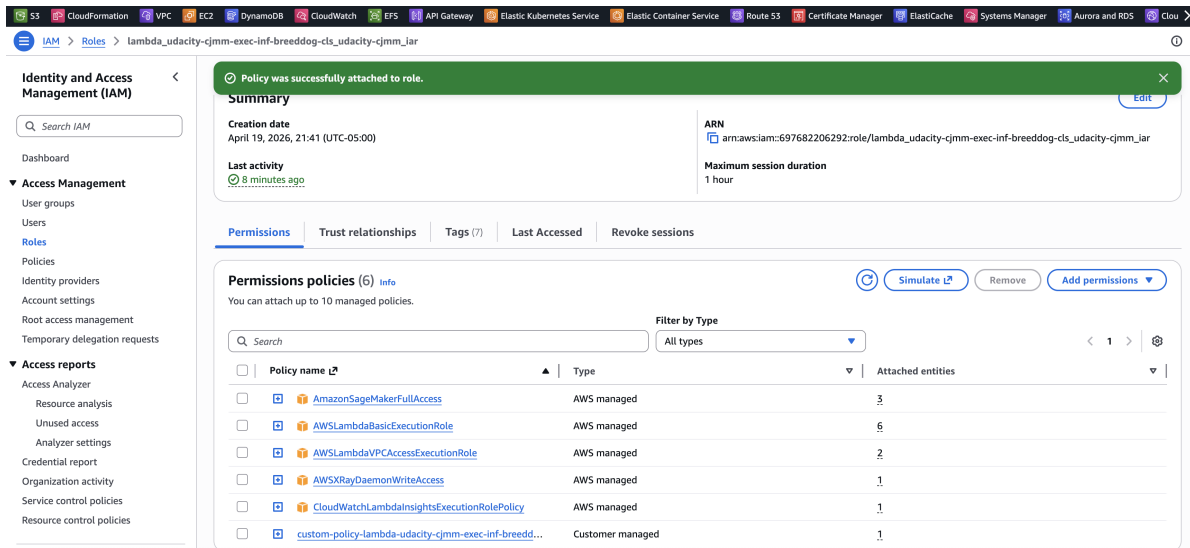


Figure 7: IAM role

Lambda function testing

The inference Lambda function was tested using the following dog breed image:



Figure 8: Carolina Dog Standing Outdoors

The following JSON request dictionary was constructed: { "url": "https://s3.amazonaws.com/cdn-origin-etr.akc.org/wp-content/uploads/2017/11/20113314/Carolina-Dog-standing-outdoors.jpg" } and the result is shown in the following screenshot:

lambda-udacity-cjmm-exec-inf-breddog-cls-dev

Throttle Copy ARN Actions

Function overview [Info](#)

Diagram Template

lambda-udacity-cjmm-exec-inf-breddog-cls-dev

Layers (0)

+ Add trigger

+ Add destination

Function ARN
arn:aws:lambda:us-east-1:697682206292:function:lambda-udacity-cjmm-exec-inf-breddog-cls-dev

Description
functionality: omnichannel generator for the master execution plan in the udacity-cjmm project

Last modified
39 minutes ago

Code Test Monitor Configuration Aliases Versions

Executing function: succeeded (logs 2)

Details

```
{
  "statusCode": 200,
  "headers": {
    "Content-Type": "text/plain",
    "Access-Control-Allow-Origin": "*"
  },
  "type-result": "eclass 'str'",
  "Content-Type-In": "LambdaContext([aws_request_id=c7dd7b-a04-4699-938b-5bb584c47f16, log_group_name=/aws/lambda/lambda-udacity-cjmm-exec-inf-breddog-cls-dev, log_stream_name=2026/04/18/[SLATEST]6d1ba2c8522477c9f7658bfc699d7a, function_name=lambda-udacity-cjmm-exec-inf-breddog-cls-dev, memory_limit_in_mb=512, function_version=SLATEST, invoked_function_arn=arn:aws:lambda:us-east-1:697682206292:function:lambda-udacity-cjmm-exec-inf-breddog-cls-dev, client_context=None, identity=CognitoIdentity([CognitoIdentityId=None, cognito_identity_pool_id=None]), tenant_id=None])",
  "body": "[[-12.12432861328125, -6.4124016761779785, -3.7209539833868848, 1.703340768814087, -4.8407816886901855, -10.343329429626465, -0.4081711769104004, 0.9049491882324219, -9.276532173156738, 0.3097786983313477, 0.44670745730400085, -6.858445167541504, -2.987046241760254, 2.740866661071773, -6.857962131500244, -3.4018287658691406, -13.97025203704834, -3.422393560409546, -3.6992363929748533, 3.8540568351745605, -7.258420944213867, -3.0604422092437744, -13.17885971069336, -9.634763717651367, -4.850439071655273, -10.00269889831543, -1.7650794982910156, -3.6084364910125732, -10.981632232660616, -2.440361976623535, -3.5564658641815186, -7.9510658634765625, -12.628043174743052, -0.2676236629486084, -14.59607095332031, -10.81665325164795, -5.96648645401001, -4.44510137976074, -0.13489195704460144, -7.043945789337158, -0.25868773460388184, -6.648404598236084, 2.741719961166382, -1.835243105883667, -0.46080427572250366, -12.266554832458496, -1.5745373964309692, 0.411499559879303, -3.8374500274658203, -1.7933127880096436, -3.2579545974731445, -9.318428039550781, -10.566427230834961, -2.9220902919769287, -4.796382904652734, -3.599449364552002, -5.4929146766626, -8.87227725982666, -3.7624080181121826, -4.133956432342529, -10.288051681518555, -10.882994651794434, -9.88753776550293, -9.804335594177246, -4.8646069795227705, -11.612247467061016, -0.8350552734940436, -8.656279830932617, -2.0532406757080053, -0.5238699913824902, 2.6478824615478516, -3.746305465668242, -4.254235744476318, -8.027582168579102, -9.95867477172852, -5.77262783085371, -14.377676963806152, -0.38458305973053, -8.321599960327148, -6.942534923553467, 4.647784233093262, -12.00125789642334, 2.988030433654785, 0.8518291711807251, -8.681705474853516, -8.481836318969727, -2.17554037094116, -9.753399848937988, -4.9097819328308105, 1.5926835536956787, -11.40121553283691, -9.719561576843262, -8.06031604042450, -11.252782821655273, -6.31536060274902, -1.540091396007412, -6.422533608703613, -6.2450611892700195, -10.094555545797363, -8.176380833190918, -10.40819606806484, -0.7889363169670105, -0.902101457118988, -6.70094633182417, -1.9407281875610352, -11.999959945678711, -6.575476169586182, 0.020139425992965698, -0.5253670811653137, -4.214991092681885, -5.079704284667969, -1.5486079454421997, -7.803822861480713, -7.587052345275879, -9.010327339172363, -1.558549165725708, -11.186843872070312, -0.14045745134353638, -7.028361797332764, 1.8088157170971436, -1.294301830200195, -4.572299803681074, -5.964436531866945, -4.02170276618457, -9.413535926818848, -3.7513742446899414, -0.783195495605469, 0.515862047672717, -6.430561965673828, -11.5747947208711, -10.314143180847168, -0.521594285649658, -8.82612133026123]]"]
```

Figure 9: Lambda function successful test

Security Considerations

The AmazonSageMakerFullAccess policy is overly permissive for the Lambda function, so a custom IAM policy was created for the Lambda with the following definition:

```
[
  {
    "Action": [
      "sagemaker:InvokeEndpoint"
    ],
    "Effect": "Allow",
    "Resource": [
      "arn:aws:sagemaker:${region}:${account_id}:endpoint/${endpoint_name}"
    ]
  },
  {
    "Effect": "Allow",
    "Action": [
      "kms:Decrypt",
      "kms:Encrypt",
      "kms:GenerateDataKey",
```

```

        "kms:DescribeKey"
    ],
    "Resource": ["${kms_key_arn}"]
}
]

```

This follows the principle of least privilege, restricting the Lambda function to only invoke the specific SageMaker endpoint it needs to execute.

Step 5: Concurrency and auto-scaling

Concurrency

In this step, to enable the Lambda function to process multiple requests simultaneously, either provisioned or reserved concurrency can be used. In this case, provisioned concurrency was configured as it is more responsive, since it keeps pre-initialized instances ready for active inference processing. However, this also increases costs.

Since high transaction volumes are not expected for these functions, it is not necessary to choose very high concurrency. Therefore, provisioned concurrency was set to a maximum capacity of 3, and reserved concurrency was set to 50.

Screenshot of lambda concurrency settings:

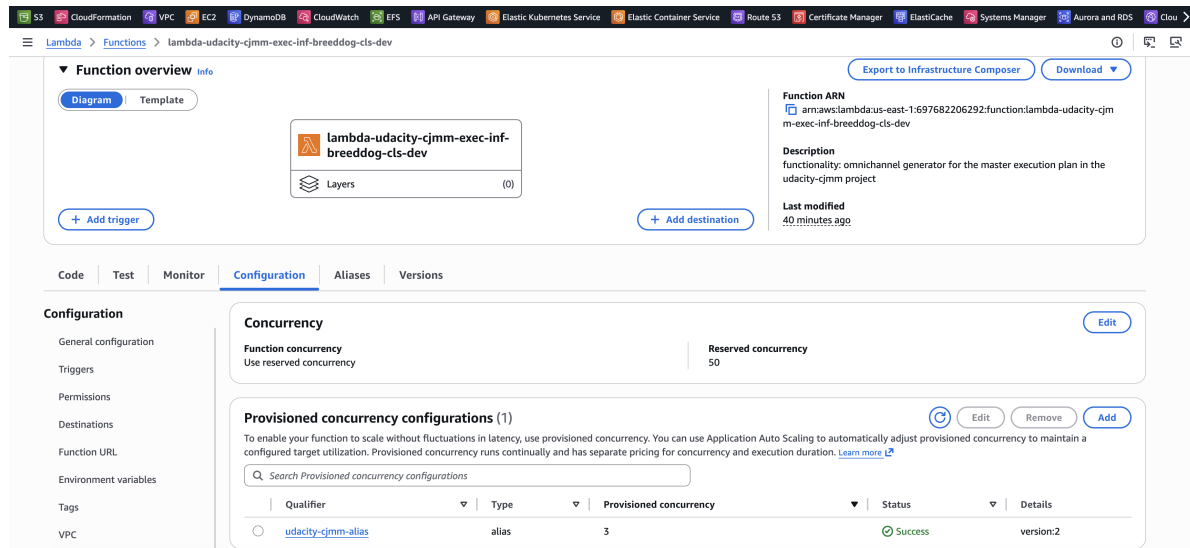


Figure 10: Lambda function concurrency settings

Auto-scaling

To support a higher volume of requests from multiple Lambda function invocations to the SageMaker endpoint, auto-scaling was configured for the endpoint to scale up to a maximum of 3 instances, with a `scale_in_cooldown` of 300 seconds and a `scale_out_cooldown` of 60 seconds. These settings were sufficient for the project needs and workload.